ML-DSA Lattice Aggregator

Compressing a Post-Quantum Validator Quorum into One Standard ML-DSA-65 Signature

Technical Whitepaper · Research Preview v0.2.0
The *lattice-aggregation* Project · July 2026
github.com/exocognosis/lattice-aggregation · MIT License

Abstract. Proof-of-stake networks rely on signature aggregation: BLS compresses thousands of validator signatures into 96 bytes, and consensus designs have grown around that property. The NIST post-quantum signature standard ML-DSA (FIPS 204) does not aggregate: its Fiat-Shamir-with-aborts structure resists the algebraic composition BLS provides, so a naive post-quantum migration multiplies consensus signature state by the validator count. This whitepaper presents the ML-DSA Lattice Aggregator, a research program and Rust artifact that studies an interactive threshold protocol whose design target is to compress a validator quorum into *one* standard-size (3,309-byte) ML-DSA-65 signature, verifiable by unmodified FIPS 204 verifiers against an epoch threshold public key. We give the mathematical preliminaries (the ML-DSA-65 parameter set, the rejection-sampling analysis, and the precise obstructions to thresholdizing Fiat-Shamir-with-aborts), define the threshold EUF-CMA security experiment, and prove the results that are provable today: injectivity of the implemented transcript encoding, information-theoretic subthreshold privacy of the share layer, and conditional correctness of aggregation. Security of the full construction is expressed as an explicit advantage decomposition, the Epsilon Residual Ledger, whose named terms ($\varepsilon_{\text{mask}}$, ε_{rej}, $\varepsilon_{\text{withhold}}$, $\varepsilon_{\text{contrib}}$, $\varepsilon_{\text{classify}}$, ε_{vss}) remain open proof obligations. We include reference-implementation listings from the audit-oriented Rust scaffold, report reproducible evidence tooling, and lay out a milestone roadmap toward theorem closure. Throughout, we distinguish design targets from proven results: the hypothesis is currently assessed *partially proven*, with all five closure criteria *partially met*.

1 Executive Summary

Blockchains are preparing to replace elliptic-curve signatures with NIST-standardized post-quantum algorithms, and the leading lattice-based standard, ML-DSA (FIPS 204, finalized August 2024), carries a hidden cost for consensus layers: it cannot be aggregated. A BLS aggregate attesting for ten thousand validators is 96 bytes; ten thousand ML-DSA-65 signatures are roughly 33 MB. Every proof-of-stake design that leans on cheap aggregation must either cap validator participation, absorb validator-count growth in bandwidth and state, or bolt a zero-knowledge proof system onto its consensus-critical verification path.

The *lattice-aggregation* project investigates a fourth option: an interactive threshold protocol in which a quorum of validators jointly produces **one standard-size ML-DSA-65 signature (3,309 bytes)** against an epoch threshold public key. If the protocol's security obligations close, verification becomes $O(1)$ in the validator count and remains byte-compatible with unmodified FIPS 204 verifiers: light clients, bridges, and hardware wallets would verify post-quantum consensus with off-the-shelf ML-DSA code, never knowing a threshold execution produced the signature.

This is a hard target. ML-DSA's rejection sampling, secret-dependent masking, and transcript-bound challenges are exactly the features that make naive threshold adaptations insecure or incompatible (Section 3 makes this precise). The field's best results to date trade away one of three properties: standard-verifier compatibility, scalability beyond a handful of signers, or practical round counts. Our contribution is a rigorously bounded research program that attacks the full combination, structured so that every gap between aspiration and proof stays visible.

Concretely, the project today comprises:

- a **formal theorem package** defining threshold EUF-CMA unforgeability and real/ideal realization targets for ML-DSA-65, with a nine-assumption adversary model, ten lemma targets, and an explicit hybrid proof plan (Section 5);
- a **proved foundation layer**: transcript-encoding injectivity, subthreshold share privacy, and conditional aggregation correctness, stated and proved in Section 5.4;
- an **Epsilon Residual Ledger** that decomposes the security distance between the threshold construction and centralized ML-DSA into named terms (ε_{vss}, $\varepsilon_{\text{mask}}$, ε_{rej}, $\varepsilon_{\text{withhold}}$, $\varepsilon_{\text{contrib}}$, $\varepsilon_{\text{classify}}$, …), each mapped to closure criteria, evidence routes, and open obligations;
- an **audit-oriented Rust scaffold** (∼92 commits, MIT-licensed): type-state signing sessions, canonical transcripts, collection validation, async actor and wire adapters, feature-gated real-ML-DSA experiment paths, and fail-closed production policy gates, excerpted as Listings 1–3 in Section 6;
- **reproducible evidence tooling**: a single assessment command regenerates the hypothesis verdict (currently *partially_proven*, all five criteria *partially_met*), and deterministic exporters regenerate checksummed benchmark artifacts.

Current status: the security theorems are targets with their proof routes mapped, and the backend today is simulation machinery plus bounded hazmat ML-DSA experiments. What distinguishes the project is that this boundary is engineered into the artifact itself rather than deferred to a disclaimer. Sections 8–9 lay out a milestone roadmap (mask/rejection distribution closure, abort-bias and partial-soundness proofs, an unforgeability reduction, comparative evaluation, external review) and the failure criteria that would trigger a pivot to fallback proof-wrapper aggregation. For a post-quantum roadmap that is deciding *now* how validator attestations will be aggregated for the next decade, a bounded, reviewable answer on the native-signature option is cheap insurance, whichever way the answer comes out.

2 Background: The Post-Quantum Aggregation Gap

2.1 Aggregation is load-bearing in proof-of-stake consensus

Modern proof-of-stake protocols assume signatures are nearly free to combine. On Ethereum, hundreds of thousands of active validators attest to blocks; BLS signatures over BLS12-381 make this tractable because

Scheme	Per-signature size	10,000-validator quorum	Post-quantum
BLS12-381 (current practice)	96 B	96 B (native aggregation)	No
Ed25519	64 B	640 KB (no aggregation)	No
ML-DSA-44 (FIPS 204, cat. 2)	2,420 B	∼24.2 MB (no aggregation)	Yes
ML-DSA-65 (FIPS 204, cat. 3)	**3,309 B**	**∼33.1 MB (no aggregation)**	**Yes**
ML-DSA-87 (FIPS 204, cat. 5)	4,627 B	∼46.3 MB (no aggregation)	Yes
This project (design target)	**3,309 B**	**3,309 B (one threshold signature)**	**Yes**

Table 1: Consensus signature footprint per block for a 10,000-validator quorum. ML-DSA sizes from FIPS 204 [1]; the final row is the project's unproven design target, not an achieved result.

pairing bilinearity lets any number of signatures compress into a single 96-byte group element that verifies in constant time [18]. Committee designs, attestation pipelines, light-client sync, and cross-chain bridge proofs are all downstream of that one algebraic property.

Lattice signatures break the assumption. ML-DSA verification checks norm bounds and hash consistency over structured polynomial vectors; these checks are nonlinear, so there is no known way to sum ML-DSA signatures and verify the sum. The consequence for a naive migration is stark (Table 1).

A 33 MB-per-block signature bill is not an engineering nuisance; it is a design constraint that forces validator-set caps, heavier committees, or a new proof system in the verification path. Each of those choices ripples into decentralization, latency, and audit surface.

2.2 The migration clock is running

The timeline pressure is institutional as much as cryptanalytic. NIST finalized FIPS 204 in August 2024 [1]. NSA's CNSA 2.0 requires exclusive use of quantum-resistant algorithms for software signing by 2030 and for all national security systems by 2035 [13]; NIST's transition guidance (IR 8547) deprecates 112-bit-security classical algorithms after 2030 and disallows them after 2035 [14]. Engineering roadmaps commonly place a cryptanalytically relevant quantum computer in the early-to-mid 2030s; and for signatures the binding constraint is not harvest-now-decrypt-later but migration lead time: validator keys are long-lived, publicly known, and secure everything; they must rotate to post-quantum schemes well before the threat materializes.

Ecosystems are already choosing. The Ethereum Foundation's post-quantum team plans to replace BLS with hash-based signatures (leanXMSS) aggregated via succinct SNARK proofs, explicitly because "BLS's native aggregation has no post-quantum equivalent" [11]; it assesses that the layer-1 upgrades could complete by 2029. Algorand has shipped Falcon signatures in state proofs since 2022 and targets broad quantum resilience by 2027 [15]. NIST's multi-party threshold cryptography call (IR 8214C, final January 2026) has opened a formal channel for exactly the class of threshold schemes studied here [4]. The window in which a native lattice aggregation result can still influence architecture decisions is open now and will not stay open long.

3 Preliminaries: ML-DSA-65 and Fiat-Shamir with Aborts

3.1 Notation

Let $q = 8380417 = 2^{23} - 2^{13} + 1$ and $n = 256$, and work in the cyclotomic ring $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$. Bold letters denote vectors and matrices over R_q. For $w \in R_q$ with coefficients in centered representation, $\|w\|_\infty$ is the largest absolute coefficient value, extended to vectors coordinate-wise. S_η is the set of ring elements with $\|w\|_\infty \le \eta$. B_τ is the challenge set: polynomials with exactly τ coefficients in $\{-1, +1\}$ and the rest zero (for $\tau = 49$, $|B_\tau| > 2^{192}$). H denotes the SHAKE-based hash/XOF family of FIPS 204, used with domain separation.

3.2 The ML-DSA-65 parameter set

Parameter	Value	Role
q	$8{,}380{,}417 = 2^{23} - 2^{13} + 1$	modulus of R_q
n	256	ring dimension
(k, ℓ)	$(6, 5)$	dimensions of $\mathbf{A} \in R_q^{k \times \ell}$
η	4	secret-key coefficient bound $(\mathbf{s}_1, \mathbf{s}_2)$
γ_1	$2^{19} = 524{,}288$	masking-vector coefficient range
γ_2	$(q-1)/32 = 261{,}888$	low-order rounding range
τ	49	± 1 coefficients in challenge c
β	$\tau \cdot \eta = 196$	max coefficient of $c \cdot \mathbf{s}_i$
ω	55	max weight of hint vector $\mathbf{h}$
Public key	$1{,}952$ B $= 32 + 6 \cdot 320$	seed ρ plus packed $\mathbf{t}_1$ (10 bits/coeff.)
Signature	$3{,}309$ B $= 48 + 5 \cdot 640 + 61$	$\tilde{c}$ (48 B) + packed $\mathbf{z}$ (20 bits/coeff.) + hint $\mathbf{h}$
Security	NIST category 3	$\approx$ AES-192; $\lambda = 192$

Table 2: ML-DSA-65 parameters (FIPS 204 [1, 2]). The byte layout explains the 3,309-byte signature the aggregator must reproduce exactly.

3.3 Signing and verification, simplified

Key generation samples $\mathbf{A} \in R_q^{6 \times 5}$ from a public seed, secrets $\mathbf{s}_1 \in S_\eta^5$, $\mathbf{s}_2 \in S_\eta^6$, and publishes $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ (compressed to $\mathbf{t}_1$). Signing is a rejection loop (Table 2 fixes the constants):

```
Algorithm 1  ML-DSA-65.Sign(sk, M)        [simplified; see FIPS 204 for exact form]
  1:  μ := H(tr || M)                       # message representative
  2:  repeat
  3:      y  <- uniform, each of the 5·256 coefficients in (−γ1, γ1]
  4:      w  := Ay;   w1 := HighBits(w, 2γ2)
  5:      c  := SampleInBall( H(μ || w1) )   # c ∈ B_τ, from 48-byte seed c̃
  6:      z  := y + c s1
  7:  until ‖z‖∞ < γ1 − β  and  ‖LowBits(w − c s2, 2γ2)‖∞ < γ2 − β
  8:  h := MakeHint(−c t0, w − c s2 + c t0)   # restart if ‖c t0‖ or wt(h) too big
  9:  return σ = (c̃, z, h)

ML-DSA-65.Verify(pk, M, σ):  recompute w1' from (A, z, c, t1, h); accept iff
  ‖z‖∞ < γ1 − β  and  c̃ = H(μ || w1')  and  wt(h) ≤ ω
```

The response $\mathbf{z}$ is released only if it lands in a norm box that is independent of the secret; otherwise the attempt is discarded and the loop restarts with a fresh mask. This is the Fiat-Shamir-with-aborts paradigm [3]: security rests on the accepted distribution of $(c, \mathbf{z}, \mathbf{h})$ carrying no information about $\mathbf{s}_1, \mathbf{s}_2$, which the rejection step enforces.

3.4 Rejection-sampling analysis

The acceptance probability of one loop iteration is a design constant worth computing, because a threshold protocol must reproduce it (or bound its deviation). Each coefficient of $c\,\mathbf{s}_1$ has absolute value at most β, so for any fixed value of that coefficient, of the $2\gamma_1$ possible mask values, exactly $2(\gamma_1 - \beta) - 1$ land $\mathbf{z}$ inside the box. Over all $256\,\ell$ coefficients:

$$\Pr\big[\|\mathbf{z}\|_\infty < \gamma_1 - \beta\big] = \left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1}\right)^{256\,\ell} \approx e^{-256\,\ell\,\beta/\gamma_1} = e^{-0.4785} \approx 0.620 \tag{1}$$

$$\Pr\big[\|\mathrm{LowBits}(\mathbf{w} - c\,\mathbf{s}_2)\|_\infty < \gamma_2 - \beta\big] \approx e^{-256\,k\,\beta/\gamma_2} = e^{-1.1496} \approx 0.317 \tag{2}$$

The two conditions are nearly independent, so a full iteration accepts with probability $\approx 0.620 \cdot 0.317 \approx 0.196$, giving an expected ≈ 5.1 signing attempts per signature, matching the Dilithium-3 design figure [2]. Two consequences matter for aggregation. First, aborts are frequent, ordinary events, not rare exceptions; a threshold protocol must handle them as part of the signing distribution. Second, the acceptance predicate is a function of the *aggregate* values $\mathbf{z}$ and $\mathbf{w} - c\,\mathbf{s}_2$, which no single party holds; in known verifier-compatible protocols the per-attempt success rate degrades further (a 2026 preprint reports 21–45% per attempt in its masked profile [10]), making abort handling a first-order design surface rather than a corner case.

3.5 Four precise obstructions to aggregation

(O1) Norm growth under summation. Verification enforces $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$. If t parties each release a locally valid response $\mathbf{z}_i$, the triangle inequality only gives

$$\Big\|\sum_{i\in S} \mathbf{z}_i\Big\|_\infty \;\le\; t\,(\gamma_1 - \beta) \tag{3}$$

which exceeds the verifier's bound by a factor of about t. Either each party must shrink its mask range by a factor of t (destroying the security margin of Equation (1)'s analysis), or masks must be coordinated so the *sum*, not each term, follows the centralized distribution. The $\varepsilon_{\text{mask}}$ term of Section 5.3 measures exactly the residual distance of that coordination.

(O2) Lagrange coefficient blowup. With Shamir-shared secrets, reconstruction over a quorum S uses field coefficients

$$\lambda_i^S \;=\; \prod_{j\in S,\, j\neq i} j\,(j-i)^{-1} \pmod q \tag{4}$$

The λ_i^S are essentially uniform in $\mathbb{Z}_q$, so $\lambda_i^S\,\mathbf{s}_{1,i}$ is not short even though $\mathbf{s}_{1,i}$ is: multiplication by a random field scalar destroys the shortness on which every norm check depends. Workable designs therefore either reshare additively per session, keep Lagrange terms inside masked or encrypted computation [8, 10], or abandon rejection sampling altogether in favor of noise flooding, which changes the verifier [5].

(O3) Partial responses leak before the abort decision. A validator's partial response $\mathbf{z}_i = \mathbf{y}_i + c\,\mathbf{a}_i$ ($\mathbf{a}_i$ its effective key contribution) is precisely the value rejection sampling exists to protect. The abort predicate (Algorithm 1, line 7) is evaluated on aggregate values, so no party can decide locally whether releasing $\mathbf{z}_i$ is safe; releasing partials for attempts that are later rejected is, in the centralized scheme, exactly the leakage the loop prevents. This forces commitment layers, masked openings, or proof-carrying contributions (the $\varepsilon_{\text{contrib}}$ obligation).

(O4) Adversarial aborts bias the accepted distribution. Because $\approx 80\%$ of attempts abort honestly (Section 3.4), a malicious participant who can cause or withhold aborts selectively can filter which candidate signatures survive, biasing the accepted distribution in a secret-dependent way unless abort behavior is proven bias-free (the $\varepsilon_{\text{withhold}}$ obligation). Combined with challenge binding (the challenge must be derived only after all commitments are fixed, Equation (5)), these four obstructions define the proof surface of Section 5.

4 Design Target and Architecture

4.1 The zero-compromise target

The project's design target is **backward-compatible verification**: the final block signature must check against the epoch threshold public key using the standard, unmodified ML-DSA-65 verification path: standard sizes (1,952-byte public key, 3,309-byte signature), standard byte encodings, standard rejection-consistent semantics. The verifier should not need to know, and should not be able to tell, that the signature came from a threshold execution. This single constraint drives every other design decision, and it is what distinguishes the effort from schemes that achieve thresholdization by changing what the verifier runs.

4.2 Epoch-based operational model

The intended L1 integration is epoch-based. At an epoch transition, the validator set runs a distributed key generation (DKG/VSS) protocol (idealized today, a production obligation tracked explicitly) to derive one epoch threshold public key with per-validator key shares. During block production, a proposer or aggregator coordinates a signing session over authenticated P2P channels:

- validators commit to masking material *before* any challenge exists (commitment-before-challenge is enforced by type-state in the implementation, Section 6);

- the challenge is derived by SHAKE256 over a canonical, domain-separated transcript binding session ID, threshold t, validator set V, public key, message, and the ordered commitment map (Equation (5), Listing 1);
- validators return proof-bound partial contribution frames; the aggregator validates each against the transcript, signer metadata, and commitment;
- the aggregator accepts a threshold-valid set (duplicates, unknown validators, threshold mismatches, and set mismatches are rejected by canonical collection rules, Listing 2), performs aggregate rejection / norm / hint checks, and emits one standard-size ML-DSA-65 signature;
- the state transition then verifies only that flat signature against the epoch key: $O(1)$ work regardless of quorum size.

The transcript and challenge derivation are fixed as

$$T = \big(\text{label} \,\|\, \text{ver} \,\|\, \text{sid} \,\|\, t \,\|\, |V| \,\|\, V \,\|\, \text{pk} \,\|\, \text{len}(m) \,\|\, m \,\|\, |\text{Com}| \,\|\, \text{Com}\big), \qquad \tilde{c} = \text{SHAKE256}(T) \tag{5}$$

with label `lattice-aggregation/threshold-mldsa65`, version 1, all counts fixed-width, and Com iterated in canonical validator order. Selective aborts, retries, timeouts, malformed frames, and equivocation are treated as first-class protocol events: they are modeled in the security framework (Section 5), surfaced as evidence-shaped artifacts by the implementation, and carried as explicit proof obligations rather than assumed away.

4.3 Profile P1: the selected production-candidate direction

Among candidate instantiations, the current operating-parameter contract (`native-threshold-mldsa65-aggregation-p1`) selects a coordinator-assisted Shamir nonce DKG profile: nonce material is secret-shared, and a coordinator role, assumed to run inside a TEE/HSM boundary in this profile, assists mask assembly and rejection decisions. The coordinator assumption is a deliberate, documented trade: it narrows the trust model in exchange for a plausible route to preserving centralized ML-DSA's masking and rejection distributions (Equations (1)–(2)), and it is precisely the kind of assumption the failure criteria (Section 9) exist to police. Contemporary work suggests the trade-space is real: recent MPC-based threshold ML-DSA achieves standard-verifier output at high round cost [8], while a 2026 preprint reports a TEE-assisted three-round profile with standard 3,309-byte output [10]. Profile P1 is a direction under evaluation, not a selected production backend.

4.4 Protocol flow and security boundaries

Figure 1 summarizes the intended end-to-end flow and annotates each stage with the ledger terms that guard it.

5 Formal Security Framework

5.1 Definitions

Definition 1 (verifier-preserving threshold signature)**.** A verifier-preserving threshold signature scheme for ML-DSA-65 is a tuple $(\text{TKeyGen}, \text{TSign}, \text{Verify})$ where $\text{TKeyGen}(1^\lambda, n, t, V)$ outputs a public key pk, key shares $\text{sk}_1, \ldots, \text{sk}_n$, and verification metadata; TSign is an interactive protocol among a subset of V holding shares, outputting σ or an abort; and Verify **is literally** MLDSA65.Verify of FIPS 204, unmodified. The last clause is the defining property: correctness and security must be achieved without touching the verifier.

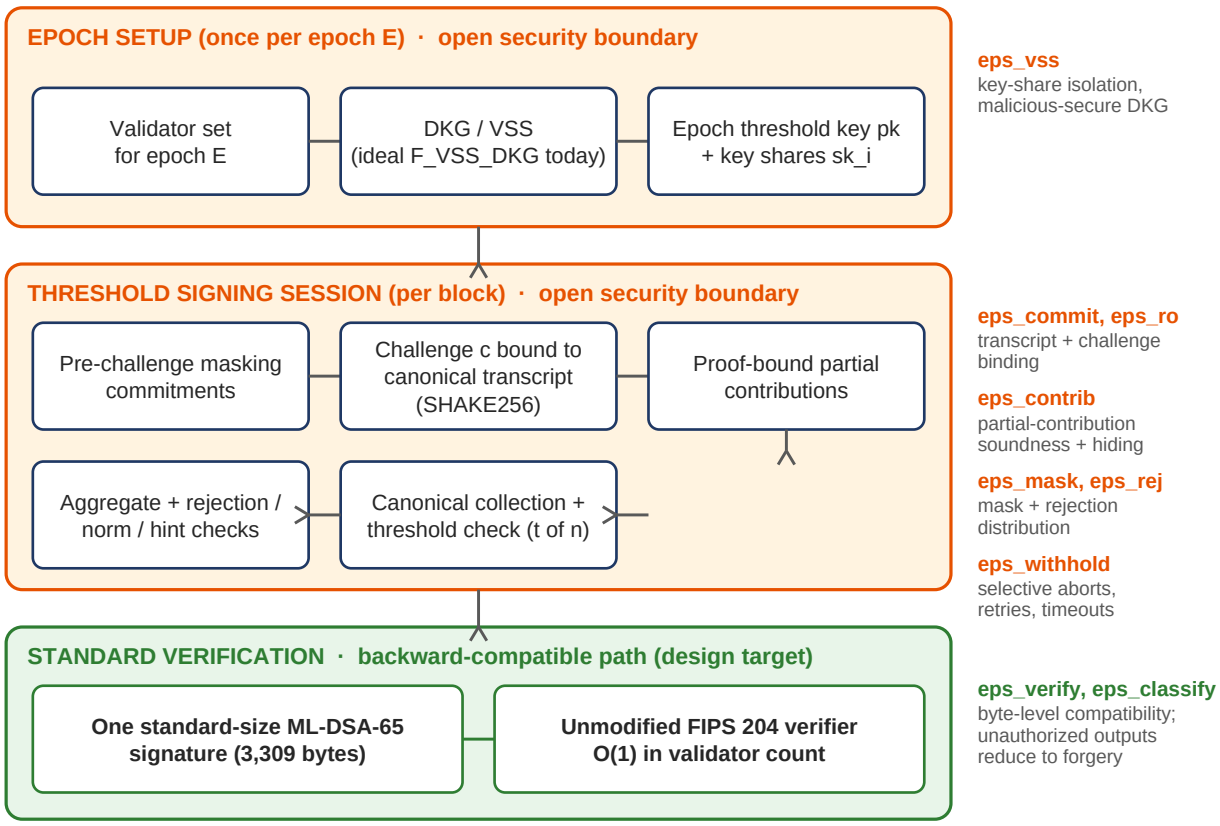

Figure 1: Intended end-to-end flow. Orange stages carry the open security boundaries of the Epsilon Residual Ledger (Section 5.3); the green stage is the backward-compatible verification path the construction must preserve. No boundary is claimed closed.

Definition 2 (threshold EUF-CMA, game FST-G1 of [16])**.** The security experiment against a static adversary corrupting fewer than t validators is the following game:

```
Game ThEUF-CMA(𝒜; λ, n, t, V)
 1:  𝒜 commits to a corruption set C ⊆ V with |C| ≤ t-1            (static)
 2:  (pk, {sk_i}, {vk_i}) <- TKeyGen(1^λ, n, t, V)
 3:  𝒜 receives pk, all vk_i, and {sk_i : i ∈ C};  Q := {}
 4:  𝒜 adaptively queries OSign(m, S):
         challenger runs an honest TSign session for m with quorum S
         (𝒜 controls scheduling, delivery, omission, duplication, and all
          corrupted parties' messages; sessions may abort);  Q := Q ∪ {m}
 5:  𝒜 outputs (m*, σ*)
 𝒜 wins iff  MLDSA65.Verify(pk, m*, σ*) = accept  and  m* ∉ Q.
 Adv(𝒜) := Pr[𝒜 wins].
```

A forgery is thus any accepting pair never authorized through the threshold functionality (FST-D6). The aggregator itself is untrusted (FST-D4): it may select arbitrary subsets of received material and attempt outputs without a valid quorum.

5.2 Adversary model and assumptions

The formal package models full control of network scheduling, delivery, omission, and duplication; static corruption of up to $t - 1$ validators before key generation; fully Byzantine behavior from corrupted validators (malformed commitments, equivocation, invalid partials, post-commitment aborts, collusion with the aggre-

Ledger term	Boundary it guards	Closure criterion	Status (v0.2.0)
$\varepsilon_{\text{mask}}$	Aggregate masking distribution matches centralized ML-DSA sampling (Eq. (1))	`aggregate_mask_distribution`: requires a Renyi-divergence bound for the selected backend	partially met
ε_{rej}	Aggregate rejection checks match centralized rejection on the same candidates (Eq. (2))	`aggregate_rejection_equivalence`: requires recomputation artifacts + reviewer sign-off	partially met
$\varepsilon_{\text{withhold}}$	Selective aborts, retries, and timeouts do not bias accepted signatures (O4)	`abort_retry_bias`: requires leakage model and bias bound across retries	partially met
$\varepsilon_{\text{contrib}}$	Every accepted partial is sound, context-bound, and hiding (O3)	`partial_contribution_soundness`: requires proof-backed local acceptance; F_{CONTRIB} idealized today	partially met
$\varepsilon_{\text{classify}}$	Every unauthorized accepting output maps to a base ML-DSA forgery or a named assumption violation	`unauthorized_aggregate_reduction`: requires threshold EUF-CMA reduction + simulator bounds	partially met
ε_{vss}	Key-share isolation through DKG/VSS; no subthreshold reconstruction (Lemma 2)	standing precondition beneath all criteria; malicious-secure DKG realization open	open (ideal $F_{\text{VSS\_DKG}}$ assumed)
$\varepsilon_{\text{commit}}$, ε_{ro}, $\varepsilon_{\text{verify}}$	Commitment binding; random-oracle domain separation; byte-level verifier compatibility	transcript-layer lemmas (FST-L1/L2, cf. Lemma 1) and a separate $\varepsilon_{\text{verify}}$ absorption decision	tests in place; formal proofs open

Table 3: The Epsilon Residual Ledger and the five hypothesis-closure criteria. The overall assessment verdict is partially_proven*; every criterion is* partially_met*.*

gator); and a signing oracle for messages of the adversary's choice. Nine explicit assumptions (FST-A1…A9) cover: base ML-DSA-65 EUF-CMA security (A1); sharing soundness, i.e. fewer than t shares yield no computationally useful information (A2); verifiable share binding (A3); commitment binding and hiding (A4); abort and noise-bound preservation (A5); partial-signature correctness and extractability (A6); transcript injectivity and domain separation (A7); canonical collection validation (A8); and constant-time implementation discipline (A9). Each assumption is either a proof obligation of this project or an imported theorem.

5.3 The Epsilon Residual Ledger

Rather than asserting security, the project maintains a conservative advantage decomposition that keeps every source of security loss visible. The target bound for any PPT adversary $\mathcal{A}$ in the game of Definition 2 is

$$\mathrm{Adv}^{\text{ThEUF-CMA}}(\mathcal{A}) \;\le\; \mathrm{Adv}^{\text{EUF-CMA}}_{\text{ML-DSA-65}}(\mathcal{B}) + \varepsilon_{\text{vss}} + \varepsilon_{\text{mask}} + \varepsilon_{\text{commit}} + \varepsilon_{\text{ro}} + \varepsilon_{\text{rej}} + \varepsilon_{\text{withhold}} + \varepsilon_{\text{contrib}} + \varepsilon_{\text{classify}} \quad (6)$$

plus implementation and audit residuals, where $\mathcal{B}$ is an explicit reduction to be constructed. Each epsilon term is a named residual with its own definition document, evidence route, and closure requirement in the repository [16]. A term is only allowed to disappear from the ledger when a proof or reviewed bound closes it. Five of the terms correspond to the project's hypothesis-closure criteria, tracked by the reproducible assessment tooling (Table 3).

5.4 What is provable today

Three foundation results can be stated and proved now. They are deliberately modest: they cover encoding, secret-sharing, and correctness facts, not the open distributional and unforgeability obligations of Table 3.

Lemma 1 (transcript-encoding injectivity)**.** *Let* $\mathrm{Enc}(\mathrm{sid}, t, V, \mathrm{pk}, m, \mathrm{Com})$ *be the byte string absorbed by Equation (5) as implemented in Listing 1, with* sid *and* pk *of fixed length, validator identifiers and counts fixed-width (16-bit), commitments fixed 32-byte,* $\mathrm{len}(m)$ *a 64-bit prefix, and* Com *serialized in canonical validator order. Then* Enc *is injective on typed tuples with* $|V|, |\mathrm{Com}| < 2^{16}$ *and* $\mathrm{len}(m) < 2^{64}$*.*

Proof. It suffices to show the encoding is uniquely parseable, since a deterministic parser inverts Enc on its image. Parse left to right: the label and version are fixed constants of known length; sid and t are fixed-width; the count $|V|$ is a fixed-width 16-bit field, after which exactly $|V|$ fixed-width identifiers follow; pk is fixed-length; $\mathrm{len}(m)$ is a fixed-width 64-bit field, after which exactly $\mathrm{len}(m)$ message bytes follow; $|\mathrm{Com}|$ is fixed-width, after which exactly $|\mathrm{Com}|$ records of fixed length $(2 + 32$ bytes) follow. At every step the number of

bytes to consume is determined by previously parsed fixed-width fields, so field boundaries are unambiguous and two distinct typed tuples cannot encode to the same string. □

Remark. Lemma 1 covers the scaffold encoding of Listing 1. The outstanding obligation FST-L1 of [16] is the same statement over the locked production transcript grammar, together with machine-checked tests; challenge binding (FST-L2) then follows in the random-oracle model for SHAKE256 with this domain separation.

Lemma 2 (subthreshold share privacy, information-theoretic)**.** *Let $s \in \mathbb{Z}_q$ be shared by Shamir's scheme [17]: sample $f(X) = s + f_1 X + \cdots + f_{t-1} X^{t-1}$ with uniform $f_1, \ldots, f_{t-1}$, and give party i the share $f(i)$. Then for any set C of at most $t-1$ parties, the joint distribution of $\{f(i)\}_{i \in C}$ is uniform on $\mathbb{Z}_q^{|C|}$ and independent of s. The same holds coefficient-wise for sharings of ring elements in R_q.*

Proof. Fix any secret s and any target share values $(v_i)_{i \in C}$ with $|C| = t - 1$. The t constraints $f(0) = s$ and $f(i) = v_i$ for $i \in C$ are interpolation conditions on t distinct points, and a polynomial of degree at most $t-1$ over the field $\mathbb{Z}_q$ is determined by exactly one assignment of its t coefficients for each such condition set (Lagrange interpolation). Hence for every s, exactly one choice of $(f_1, \ldots, f_{t-1})$ realizes each value vector, so each of the q^{t-1} value vectors occurs with probability $q^{-(t-1)}$ regardless of s. Independence follows. □

Remark. Lemma 2 grounds assumption FST-A2 for an honest dealer or ideal $F_{\text{VSS\_DKG}}$. It says nothing about what *released partial responses* reveal: $\mathbf{z}_i$ is a function of both the share and the mask, and bounding that leakage is exactly the open $\varepsilon_{\text{contrib}}$ and $\varepsilon_{\text{mask}}$ work, not a corollary of this lemma. Realizing the dealerless, malicious-secure DKG is the open ε_{vss} obligation.

Lemma 3 (conditional correctness of aggregation, FST-L5 shape)**.** *Fix a session with quorum S, $|S| = t$, common transcript challenge c (Equation (5)), and an effective additive decomposition of the signing secret $\mathbf{s}_1 = \sum_{i \in S} \mathbf{a}_i$ (obtained from per-session additive re-sharing or backend-internal masked Lagrange terms $\mathbf{a}_i = \lambda_i^S \mathbf{s}_{1,i}$). Suppose each participant outputs $\mathbf{z}_i = \mathbf{y}_i + c\,\mathbf{a}_i$ and the aggregate mask is $\mathbf{y} = \sum_{i \in S} \mathbf{y}_i$. Then*

$$\mathbf{z} \;=\; \sum_{i \in S} \mathbf{z}_i \;=\; \mathbf{y} + c\,\mathbf{s}_1 \tag{7}$$

i.e. the aggregate response has exactly the algebraic form of a centralized ML-DSA response for mask $\mathbf{y}$. If, additionally, the assembled candidate $(\tilde{c}, \mathbf{z}, \mathbf{h})$ satisfies the standard acceptance predicates of Algorithm 1 (norm bound, low-bits bound, hint weight, challenge recomputation), then MLDSA65.Verify$(\text{pk}, M, (\tilde{c}, \mathbf{z}, \mathbf{h}))$ *accepts.*

Proof. Equation (7) is R_q-linearity: addition of ring vectors and multiplication by the common challenge c distribute over the sums defining $\mathbf{y}$ and $\mathbf{s}_1$. For the second claim, observe that MLDSA65.Verify is a deterministic function of $(\text{pk}, M, \tilde{c}, \mathbf{z}, \mathbf{h})$ alone; it does not reference how the candidate was produced. A candidate of the form $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}_1$ that satisfies the signer-side acceptance predicates satisfies the verifier-side recomputation by the single-signer correctness argument of FIPS 204 [1], which depends only on these predicates and the key equation $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$. □

Remark. Lemma 3 isolates what is genuinely hard: correctness is essentially free, while *security* requires that the accepted distribution of $(\mathbf{z}, \mathbf{h})$ and the abort behavior match centralized signing ($\varepsilon_{\text{mask}}$, ε_{rej}, $\varepsilon_{\text{withhold}}$) and that every accepted partial is sound and hiding ($\varepsilon_{\text{contrib}}$). Those are the open obligations of Table 3, not consequences of linearity.

5.5 Theorem targets and proof plan

Theorem 1 (target FST-T1, threshold unforgeability)**.** *Under assumptions FST-A1…A8, for any PPT adversary corrupting at most $t-1$ validators,* $\mathrm{Adv}^{\mathrm{ThEUF\text{-}CMA}}(\mathcal{A})$ *is negligible in λ, with the concrete bound of Equation (6).* ***Status: open.***

The immediate route is the conditional variant FST-T1-IdealVSS, which assumes ideal setup ($F_{\mathrm{VSS\_DKG}}$) and ideal contribution validation (F_{CONTRIB}) and is assembled from lemma batches FST-L1…L7 and FST-L10 of [16]; Lemmas 1–3 above are the scaffold-level instances of the L1/L5 layer.

Theorem 2 (target FST-T2, real/ideal realization)**.** *Under FST-A1…A9, the production protocol UC-realizes the ideal threshold signing functionality F_{TMLDSA} against static Byzantine corruption of at most $t-1$ validators, in the random-oracle model selected for ML-DSA-65.* ***Status: open****; requires a complete simulator plus distinguishing bounds for the hybrid chain below.*

```
H0  real protocol (real DKG, commitments, partials, aggregation, scheduling)
H1  replace network delivery with ideal scheduling            [bound: open]
H2  replace rejected malformed/duplicate/invalid messages
    with ideal evidence events                                [bound: open]
H3  replace honest partial-share generation with simulated
    shares; accepting aggregates only via ideal signing       [bound: open]
H4  replace aggregate signatures with F_TMLDSA.Sign outputs   [bound: open]
H5  ideal execution; any unauthorized accepting output implies
    an ML-DSA-65 forgery or a named assumption violation      [classifier]
```

Sketch of the intended reduction (H5 classifier route). Given an adversary that outputs an unauthorized accepting (m^*, σ^*), the classifier of [16] partitions executions into ordered cases: transcript or challenge collision (contradicts Lemma 1 plus the random-oracle bound $\varepsilon_{\mathrm{ro}}$); subthreshold key reconstruction (contradicts Lemma 2 under ideal setup, else breaks FST-A2); unsound or replayed partial contribution (charged to $\varepsilon_{\mathrm{contrib}}$ via F_{CONTRIB} extraction); abort-biased acceptance (charged to $\varepsilon_{\mathrm{withhold}}$); and a residual case in which the simulator embeds an external ML-DSA-65 public key and returns (m^*, σ^*) as a forgery against assumption FST-A1. The open work, tracked as $\varepsilon_{\mathrm{classify}}$, is proving the case analysis total and disjoint with residual $\varepsilon_{\mathrm{cls\_unmapped}} = 0$. This sketch is a proof plan, not a proof.

Theorem 3 (target FST-T3, transcript non-malleability)**.** *No adversary can bind one partial or aggregate signature to two distinct typed transcripts; partially supported today by deterministic binding tests plus Lemma 1, with the formal encoding-level proof over the production grammar open.*

Theorem FST-T4 of [16] is a traceability statement: passing the conformance suite is necessary but never sufficient evidence for Theorems 1–2.

6 Reference Implementation

6.1 Crate architecture and selected listings

The artifact is a single Rust crate (`lattice-aggregation`, edition 2021, `#![forbid(unsafe_code)]`) organized around auditable boundaries: `transcript`, `collections`, `protocol`, `aggregation`, `dkg`, `backend`, `low_level/poly`, and `adapter` (async actors, wire formats, evidence). Dependencies are deliberately spare: the RustCrypto `ml-dsa` crate (optional, feature-gated), `sha3`, `serde`, `tokio`, `zeroize`. Listing 1 is the implementation of Equation (5); every byte absorbed into the challenge is visible in one function.

Collection validation enforces assumption FST-A8 (and Lemma FST-L3's validator-set soundness) at the type level: a `CommitmentSet` cannot be constructed with duplicates, unknown validators, an invalid threshold, or fewer than t entries (Listing 2).

The signing round structure is enforced by type-state: a session in state `Initialized` can only emit a commitment; only a session holding all bound commitments can derive the challenge and emit a partial signature;

```
// src/transcript.rs (excerpt)
const PROTOCOL_LABEL: &[u8] = b"lattice-aggregation/threshold-mldsa65";
const PROTOCOL_VERSION: u16 = 1;

fn derive_challenge(
    session_id: SessionId, threshold: u16, validator_set: &[ValidatorId],
    public_key: &ThresholdPublicKey, message: &[u8], commitments: &CommitmentSet,
) -> Challenge {
    let mut hasher = Shake256::default();
    hasher.update(PROTOCOL_LABEL);
    hasher.update(&PROTOCOL_VERSION.to_be_bytes());
    hasher.update(&session_id);
    hasher.update(&threshold.to_be_bytes());
    hasher.update(&(validator_set.len() as u16).to_be_bytes());
    for id in validator_set {
        hasher.update(&id.0.to_be_bytes());
    }
    hasher.update(&public_key.0);
    hasher.update(&(message.len() as u64).to_be_bytes());
    hasher.update(message);
    hasher.update(&(commitments.len() as u16).to_be_bytes());
    for (id, commitment) in commitments.iter() {   // canonical validator order
        hasher.update(&id.0.to_be_bytes());
        hasher.update(&commitment.0);
    }
    let mut reader = hasher.finalize_xof();
    let mut out = [0u8; 32];
    reader.read(&mut out);
    Challenge(out)
}
```

Listing 1: Domain-separated challenge derivation over the canonical transcript (`src/transcript.rs` [16]). Length prefixes and canonical iteration order are the code-level counterpart of Lemma 1.

```
// src/collections.rs (excerpt)
impl CommitmentSet {
    pub fn new(
        validators: Vec<ValidatorId>, threshold: u16,
        commitments: Vec<(ValidatorId, Commitment)>,
    ) -> Result<Self, ThresholdError> {
        validate_threshold(threshold, validators.len() as u16)?;   // 1 <= t <= n
        let validator_set = set_from_validators(validators)?;      // rejects duplicates

        let mut ordered = BTreeMap::new();                         // canonical order
        for (validator, commitment) in commitments {
            if !validator_set.contains(&validator) {
                return Err(ThresholdError::UnknownValidator { validator });
            }
            if ordered.insert(validator, commitment).is_some() {
                return Err(ThresholdError::DuplicateValidator { validator });
            }
        }
        if ordered.len() < threshold as usize {
            return Err(ThresholdError::InsufficientCommitments {
                required: threshold, received: ordered.len(),
            });
        }
        Ok(Self { validator_set, threshold, commitments: ordered })
    }
}
```

Listing 2: Canonical collection validation (`src/collections.rs` [16]). `BTreeMap` iteration makes network arrival order irrelevant to the transcript, closing one replay/grinding surface by construction.

```
// src/protocol.rs and src/aggregation.rs (excerpts)
pub trait ThresholdSigner: Sized {
    type Error;
    /// Generate the local commitment and advance to commitment collection.
    fn initiate_signing(self)
        -> Result<(SigningSession<state::AwaitingCommitments>, Commitment), Self::Error>;
    /// Bind all commitments, derive the challenge, emit the local partial signature.
    fn generate_partial_signature(
        session: SigningSession<state::AwaitingCommitments>,
        all_commitments: CommitmentSet, message: &[u8],
    ) -> Result<(SigningSession<state::AwaitingPartialSignatures>,
                 PartialSignatureShare), Self::Error>;
}

impl SignatureAggregator for SimulatedAggregator {
    type Error = ThresholdError;
    fn aggregate_shares(
        transcript: ThresholdSigningTranscript, partial_shares: PartialShareSet,
    ) -> Result<ThresholdSignature, Self::Error> {
        if partial_shares.threshold() != transcript.threshold()
            || !partial_shares.validators().iter().copied()
                .eq(transcript.validator_set().iter().copied())
        {
            return Err(ThresholdError::TranscriptMismatch);
        }
        SimulatedBackend::aggregate(transcript.public_key(), &transcript, partial_shares)
    }
}
```

Listing 3: The type-state signing interface and the aggregation boundary (`src/protocol.rs`, `src/aggregation.rs` [16]). The `SimulatedBackend` shown here is deterministic test machinery; real ML-DSA-65 paths live behind the feature-gated hazmat provider.

the commitment-before-challenge discipline of Section 4.2 is therefore unrepresentable to violate in safe Rust. Aggregation re-checks transcript agreement before combining anything (Listing 3).

Feature gates partition the risk surface explicitly: `protocol-sim` (deterministic simulation), `raw-mldsa-internals` and `raw-real-mldsa` (hazmat ML-DSA-65 experimentation with KAT-style fixtures and differential checks against standard verification), `coordinator-assisted`, and `production-mldsa65-coordinator` (the Profile P1 boundary under review). Simulated paths produce ML-DSA-65-*sized* outputs for protocol testing; they are not cryptographic instantiations, and the code says so in its types and documentation.

6.2 Fail-closed production policy

A distinctive property of the scaffold: production-labeled configurations **reject scaffold backends in code**. Policy gates refuse to run simulation or unreviewed-backend families under a production label, so research machinery cannot quietly masquerade as production security: the claim boundary is enforced by the compiler and the configuration validator, not by a README warning alone. Evidence paths (malformed-frame handling, retry telemetry, equivocation records) emit artifacts shaped for later slashing integration without claiming chain-specific fraud proofs.

6.3 Reproducible evidence

Every reviewer-facing claim is regenerable. `scripts/assess_lattice_hypothesis.py` recomputes the hypothesis verdict from the checked-in evidence (currently *partially_proven*, five criteria *partially_met*); `scripts/reproduce-section-v.sh` regenerates the benchmark and transcript artifact bundle and checks it against committed SHA-256 checksums; documentation manifest tests validate that every claim document, link, and anchor referenced by the proof ledger exists. Continuous checks (rustfmt, clippy with denied warnings, full test suite across all features) gate the tagged releases (v0.1.0 protocol conformance; v0.2.0-research-preview grant readiness and Ethereum/PQ alignment).

Scheme	Rounds	Output	Std. ML-DSA verifier	Scale	Notes
Threshold Raccoon (Eurocrypt'24) [5]	3	∼13 KiB Raccoon sig	No	t up to 1024	First efficient lattice TS; avoids aborts via noise flooding
Ringtail (IEEE S&P'25) [6]	2 (1 offline)	∼13.4 KB	No	∼1024	Standard LWE; WAN deployment demonstrated
Threshold Sigs Reloaded (2025) [7]	interactive	standard ML-DSA	Yes	up to 6 signers	First practical ML-DSA-compatible TS; small quorums
Quorus / MPC ML-DSA (2025) [8]	many (MPC)	standard ML-DSA	Yes	arbitrary; honest majority	∼100 KB per party per rejection round; no new assumptions
Trilithium (2025) [9]	2-party	standard ML-DSA	Yes	$n = 2$	Server + phone; UC-secure; FIPS 204-compliant
Masked-Lagrange preprint (2026) [10]	3, with TEE	standard 3,309 B	Yes	arbitrary (claimed)	arXiv preprint; not peer-reviewed
This project (target)	epoch DKG + interactive session	**standard 3,309 B**	**Yes (target)**	**validator scale (target)**	Unproven; five criteria open; coordinator-assisted P1 profile under evaluation

Table 4: The threshold-lattice trade-space. No published scheme yet combines standard-verifier compatibility, validator-scale quorums, and low round count without added trust assumptions or heavy MPC; rows summarize the cited works' own reported properties.

7 Related Work and Positioning

7.1 Threshold lattice signatures

Threshold lattice signing has moved fast since 2024, and the results map the trade-space this project targets (Table 4).

The pattern is a three-way trade among verifier compatibility, quorum scale, and interaction cost, and it is a direct consequence of obstructions O1–O4: schemes that scale (Raccoon, Ringtail) change the verifier to escape rejection sampling; schemes that keep the verifier either cap the quorum (six signers), pay heavy MPC costs per rejection round, or add hardware trust. This project's bet is that a coordinator-assisted profile with rigorously bounded residuals can reach validator-scale quorums while keeping the standard verifier, and its Epsilon Residual Ledger is designed to reveal, rather than obscure, exactly what that bet costs if it fails.

7.2 Relation to Ethereum's hash-based + SNARK path

Ethereum's post-quantum roadmap aggregates hash-based signatures with succinct proofs (leanXMSS + a minimal zkVM) [11, 12], a fundamentally different route: prove many signatures inside a proof system versus produce one native signature. The approaches are complementary, not competing. The SNARK path buys aggregation generality at the cost of a prover/verifier stack inside consensus; the native path buys a minimal verification base (one standardized signature check) at the cost of hard, currently open distribution and soundness proofs. A robust ecosystem roadmap benefits from a bounded, reviewable evaluation of both. Even a negative result here ("the native option costs X in assumptions") directly informs the choice, by identifying which obligations gate *any* native post-quantum aggregator.

7.3 Fallback: proof-wrapper aggregation

If native threshold proof closure stalls, the documented fallback is Falcon/LaBRADOR-style proof aggregation: proving many independent signatures under a lattice-native proof system. It is held at *evaluate only* status: any pivot would require its own scheme selection, prover and verifier benchmarks, consensus-latency analysis, and fresh claim-boundary documentation. Keeping the fallback explicit is part of the project's risk discipline (Section 9).

#	Milestone	Key deliverables	Est.
M1	Mask & rejection distribution closure	Renyi-divergence evidence for $\varepsilon_{\text{mask}}$; real (non-fixture) aggregate-rejection recomputation closing the Criterion 2 payload	4–6 mo
M2	Abort-bias & partial-soundness closure	Concrete retry/timeout policy with an accepted-sample bias bound; production local-acceptance and partial-verification predicates with soundness and hiding evidence (the long pole)	6–12 mo
M3	Unforgeability reduction & theorem assembly	Per-case reductions completed; threshold EUF-CMA reduction to base ML-DSA forgery or a named threshold assumption; assessment verdict moves beyond *partially_proven*	3–5 mo
M4	Comparative evaluation & reference spec	Apples-to-apples comparison vs. hash-based + SNARK aggregation (verification cost, liveness, trust, audit surface); reference protocol spec + conformance suite	2–3 mo
M5	External review & audit preparation	Independent cryptographic review of M1–M3; side-channel and randomness review scope; malicious-secure DKG realization plan	3–4 mo

Table 5: Milestone roadmap. Foundational proof-route scaffolding, reproducible assessment, and fail-closed release boundaries already exist, so effort concentrates on closing criteria and external review.

8 Roadmap: From Scaffold to Theorem Closure

The remaining work is theorem closure, not more scaffolding. Milestones are scoped so each ends with an externally checkable deliverable; estimates are planning-grade for an experienced cryptographer. M1–M3 close the five hypothesis criteria; M4–M5 run alongside and after (Table 5).

Nearer-term technical sequencing, per the proof-gap priority map: lock the production transcript grammar so random-oracle, contribution, evidence, and classifier proofs share one canonical byte-level input language (upgrading Lemma 1 to its production form); convert the IdealVSS route into lemma-by-lemma proof text (transcript injectivity, challenge binding, collection soundness first); keep F_{CONTRIB} idealized until a concrete contribution-proof or MPC backend theorem is selected; and carry $\varepsilon_{\text{verify}}$ separately until byte-level verifier and rejection proofs close over the same candidate tuple.

9 Risk Register and Failure Criteria

A credible research program states in advance what failure looks like. The operating contract defines the native threshold path as failed or blocked if any of the following cannot be repaired within Profile P1 assumptions:

- aggregate masks are distinguishable from the centralized ML-DSA-65 sampling model beyond the reviewed bound (Equation (1)'s distribution);
- accepted threshold outputs fail standard ML-DSA-65 verification or require a non-standard verifier;
- retry timing, abort behavior, or attempt binding biases accepted signatures beyond the reviewed proof bound;
- stale, cross-context, malformed, or leaking partial contributions can pass local or aggregate acceptance;
- an unauthorized accepting aggregate cannot be classified as a base ML-DSA forgery or a named threshold-side assumption violation.

Failure is declared only on reviewed evidence, and triggers the documented fallback evaluation (Section 7.3). Beyond the cryptographic criteria, we track: the **TEE/HSM coordinator assumption** in Profile P1 (a real trust addition relative to pure-cryptographic paths; part of what M4's comparative evaluation must price); **adaptive corruption** (the base theorems target static corruption; adaptive security needs erasure semantics and a separate state-exposure model); **side channels** (no constant-time audit or randomness review is complete, and assumption FST-A9 makes implementation discipline an explicit proof dependency); and **ecosystem timing** (Ethereum's roadmap is advancing on the hash-based + SNARK path; this project's value holds either way, but its window for influencing architecture narrows over time).

10 Conclusion

Post-quantum migration will force every proof-of-stake network to answer the same question: when BLS aggregation goes away, what compresses a validator quorum? The *lattice-aggregation* project pursues the most conservative possible answer at the verifier (one standard ML-DSA-65 signature, checked by unmodified FIPS 204 code) and the most demanding possible answer at the prover, where masking, rejection, abort, and soundness distributions must all be preserved across a distributed execution. The artifact shipped today is a research scaffold with an unusual property: its security gaps are enumerated, named, bounded in a ledger, wired into reproducible assessment tooling, and enforced by fail-closed release gates; its provable foundation (Lemmas 1–3) is separated cleanly from its open obligations (Equation (6)). The hypothesis stands at *partially proven*. The roadmap to close it is concrete, milestone-scoped, and openly falsifiable, and both outcomes carry value: a proven native aggregator would remove the largest scaling penalty in post-quantum consensus, while a well-documented negative result would tell every post-quantum roadmap exactly which assumptions that penalty buys back. We invite cryptographers, protocol engineers, and reviewers to engage with the proof ledger, challenge the boundaries, and help close, or falsify, the remaining criteria.

Project: github.com/exocognosis/lattice-aggregation (MIT). Maintainer: Rick Glenn (GitHub: exocognosis). Co-maintainers and reviewers from cryptography and post-quantum research groups are welcome; see AUTHORS.md and the reviewer quickstart in the repository's docs.

References

[1] NIST, "FIPS 204: Module-Lattice-Based Digital Signature Standard," August 13, 2024. https://csrc.nist.gov/pubs/fips/204/final

[2] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé, "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme," IACR TCHES 2018(1).

[3] V. Lyubashevsky, "Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures," ASIACRYPT 2009.

[4] NIST, "IR 8214C: NIST First Call for Multi-Party Threshold Schemes," final, January 20, 2026. https://csrc.nist.gov/projects/threshold-cryptography

[5] R. del Pino, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, M.-J. Saarinen, "Threshold Raccoon: Practical Threshold Signatures from Standard Lattice Assumptions," EUROCRYPT 2024. https://eprint.iacr.org/2024/184

[6] C. Boschini, D. Kaviani, R. W. F. Lai, G. Malavolta, A. Takahashi, M. Tibouchi, "Ringtail: Practical Two-Round Threshold Signatures from Learning with Errors," IEEE S&P 2025. https://eprint.iacr.org/2024/1113

[7] G. Borin, S. Celi, R. del Pino, T. Espitau, G. Niot, T. Prest, "Threshold Signatures Reloaded: ML-DSA and Enhanced Raccoon with Identifiable Aborts," 2025. https://eprint.iacr.org/2025/1166

[8] A. Bienstock, L. de Castro, D. Escudero, A. Polychroniadou, A. Takahashi, "Threshold ML-DSA: An MPC Approach (Quorus)," 2025. https://eprint.iacr.org/2025/1163

[9] A. Dufka, D. Kravtšenko, P. Laud, N. Snetkov, "Trilithium: Efficient and Universally Composable Distributed ML-DSA Signing," 2025. https://eprint.iacr.org/2025/675

[10] "FIPS 204-Compatible Threshold ML-DSA via Masked Lagrange Reconstruction," arXiv:2601.20917, 2026. Preprint; not peer-reviewed.

[11] Ethereum Foundation Post-Quantum Team, "Ethereum Post-Quantum Roadmap," 2026. https://pq.ethereum.org

[12] J. Drake, D. Khovratovich, M. Kudinov, B. Wagner, "Hash-Based Multi-Signatures for Post-Quantum Ethereum," IACR Communications in Cryptology, 2025. https://eprint.iacr.org/2025/055

[13] NSA, "Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) FAQ," updated 2025.

[14] NIST, "IR 8547 (Initial Public Draft): Transition to Post-Quantum Cryptography Standards," November 2024.

[15] Algorand, "Quantum-Resistant Transactions on Algorand with Falcon Signatures," technical brief, 2025.

[16] The lattice-aggregation Project, repository and proof-route documentation (README; formal-security-theorem.md; thesis-operating-parameters.md; epsilon-residual-ledger-final-form.md; proof-closure-ledger.md; protocol-flow.md; grant one-pager; src/), v0.2.0-research-preview, 2026. https://github.com/exocognosis/lattice-aggregation

[17] A. Shamir, "How to Share a Secret," Communications of the ACM 22(11), 1979.

[18] D. Boneh, B. Lynn, H. Shacham, "Short Signatures from the Weil Pairing," ASIACRYPT 2001.

Document status: this whitepaper describes research targets and an artifact boundary current as of v0.2.0-research-preview (July 2026). Sizes and external facts are cited to their sources. The proofs in Section 5.4 cover encoding, secret-sharing, and conditional-correctness facts; all other security-relevant statements about the construction are targets unless a proof document in the repository states otherwise. Algorithm 1 is a simplified presentation; FIPS 204 is normative. Sections 2–3 include contextual claims sourced from third parties [1–15]; reference [10] is an unreviewed preprint. Code listings are excerpts of the cited files at the referenced tag, lightly reformatted for layout.